```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
uploaded = files.upload()

video_path = list(uploaded.keys())[0]
print("Video selected:", video_path)
```

```
Choose Files 13064646_..._30fps.mp4
13064646_1080_1920_30fps.mp4(video/mp4) - 9513639 bytes, last modified: 9/20/2026 - 100% done
Saving 13064646_1080_1920_30fps.mp4 to 13064646_1080_1920_30fps.mp4
Video selected: 13064646_1080_1920_30fps.mp4
```

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

cap = cv2.VideoCapture(video_path)

# Read first frame
ret, old_frame = cap.read()

if not ret:
    print("Error: Could not read video.")
else:
    old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

    # Shi-Tomasi corner detection
    old_points = cv2.goodFeaturesToTrack(
        old_gray,
        maxCorners=100,
        qualityLevel=0.3,
        minDistance=7,
        blockSize=7
    )

    # Parameters for Lucas-Kanade optical flow
    lk_params = dict(
        winSize=(15, 15),
        maxLevel=2,
        criteria=(
            cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,
            10,
            0.03
        )
    )

    # Random colors for drawing trajectories
    color = np.random.randint(0, 255, (100, 3))

    # Mask for drawing trajectories
    mask = np.zeros_like(old_frame)

    frame_count = 0

    while True:
        ret, frame = cap.read()

        if not ret:
            break

        frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Calculate optical flow
        new_points, status, error = cv2.calcOpticalFlowPyrLK(
            old_gray,
            frame_gray,
            old_points,
```

```
            None,
            **lk_params
        )

        if new_points is None:
            break

        # Select good points
        good_new = new_points[status == 1]
        good_old = old_points[status == 1]

        # Draw trajectories
        for i, (new, old) in enumerate(zip(good_new, good_old)):
            a, b = new.ravel()
            c, d = old.ravel()

            a, b, c, d = map(int, [a, b, c, d])

            mask = cv2.line(
                mask,
                (a, b),
                (c, d),
                color[i].tolist(),
                2
            )

            frame = cv2.circle(
                frame,
                (a, b),
                5,
                color[i].tolist(),
                -1
            )

        output = cv2.add(frame, mask)

        # Display every 10th frame
        if frame_count % 10 == 0:
            cv2_imshow(output)

        old_gray = frame_gray.copy()
        old_points = good_new.reshape(-1, 1, 2)

        frame_count += 1

        # Stop after 100 frames for Colab
        if frame_count >= 100:
            break

    cap.release()

print("Lucas-Kanade tracking completed.")
```


Poczta















